

Today 11:53 AM

I am doing a Vibe Coding lab for a Tic-Tac-Toe project.

I have uploaded the original project files, including:

- main.py
- game_engine.py
- rules.py
- renderer.py
- ai.py
- README.md

Please inspect the existing code carefully before making changes.

I need to complete TASK 1 ONLY: Fix win and draw detection.

The README specifies these three problems:

1. Diagonal wins are not detected.
2. If the winning move also fills the last empty cell, the game reports "Draw!" instead of the actual winner.
3. After a round has ended, the player can still click empty cells and place new symbols.

Please fix all three problems.

Requirements:

- Both diagonals must count as winning lines.
- A winner must always be checked before declaring a draw.
- Once the round has ended, further board clicks must be ignored.
- Preserve the existing game architecture and behavior as much as possible.
- Do not implement Task 2 (scoreboard).
- Do not implement Task 3 (occupied-cell validation).
- Do not implement Task 4 (first-player choice or separate restart controls).
- Do not make unrelated changes.

First explain which files need to be changed and why.
Then provide the exact code changes for Task 1.

Worked for 8s

I inspected the existing `rules.py` and `game_engine.py`, and also checked the project structure described in the README.

Files that need to change

Only 2 files need changes for Task 1:

1. `game/rules.py`

This is where winning lines are defined.

Current problem:

- It checks 3 rows.
- It checks 3 columns.
- It **does not check either diagonal**.

So we need to add:

- top-left → bottom-right
- top-right → bottom-left

2. `game/game_engine.py`

This controls the round state and decides whether the game has ended.

There are two Task 1 fixes here:

Problem 2 — Win vs Draw

Currently it does:

Python

```
if is_board_full(self.board):  
    ...  
    return  
  
winner = check_winner(self.board)
```

That means a full board is declared a draw **before** checking for a winner.

We must reverse the order:

</> Python

```
winner = check_winner(self.board)
if winner:
    ...
    return

if is_board_full(self.board):
    ...
```

Problem 3 — Clicks after round ends

`handle_click()` currently doesn't check `self.round_over`, so after the game finishes, clicks can still place symbols.

We need to add this at the beginning:

</> Python

```
if self.round_over:
    return
```

Files that should NOT change

For Task 1, do **not** modify:

- `main.py`
- `renderer.py`
- `ai.py`
- `README.md`

Also, we will **not** add the occupied-cell check because that belongs to **Task 3**.

Exact changes

Change 1 — `game/rules.py`

Find this:

</> Python

```
def check_winner(board):
    lines = []
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])
    # NOTE: diagonals are not included here

    for line in lines:
        if line[0] is not None and line[0] == line[1] == line[2]:
            return line[0]
    return None
```

Replace it with:

</> Python

```
def check_winner(board):
    lines = []
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])

    # Both diagonals
    lines.append([board[0][0], board[1][1], board[2][2]])
    lines.append([board[0][2], board[1][1], board[2][0]])

    for line in lines:
        if line[0] is not None and line[0] == line[1] == line[2]:
            return line[0]
    return None
```

This gives the game all **8 winning lines**:

- 3 rows
- 3 columns
- 2 diagonals

Change 2 — game/game_engine.py

A. Ignore clicks after the round ends

Find:

</> Python

```
def handle_click(self, pos):
    if self.current_player != HUMAN_SYMBOL:
        return # not your turn - the computer is about to move (or already has)
```

Change it to:

</> Python

```
def handle_click(self, pos):
    if self.round_over:
        return

    if self.current_player != HUMAN_SYMBOL:
        return # not your turn - the computer is about to move (or already has)
```

Do not add an occupied-cell check here. That would be Task 3.

B. Check winner before draw

Find the current `check_round_end()` :

</> Python

```
def check_round_end(self):
    if is_board_full(self.board): # BUG: checked before looking for a winner
        self.round_over = True
        self.winner = None
        return
    winner = check_winner(self.board)
    if winner:
        self.round_over = True
        self.winner = winner
```

Replace it with:

</> Python

```
def check_round_end(self):
    winner = check_winner(self.board)
    if winner:
        self.round_over = True
        self.winner = winner
        return

    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
```

The important part is:

`</>` Python

```
winner = check_winner(self.board)
```

comes before:

`</>` Python

```
is_board_full(self.board)
```

Final Task 1 result

After these changes:

Situation	Result
X gets 3 horizontally	X wins!
X gets 3 vertically	X wins!
X gets top-left → bottom-right	X wins!
X gets top-right → bottom-left	X wins!
O gets a diagonal	O wins!
Winning move fills the last cell	Winner is shown, not Draw

Situation	Result
Click after round ends	Ignored
Occupied-cell click	Not changed yet — Task 3
Scoreboard	Not added — Task 2
First-player choice	Not added — Task 4

So **only** `game/rules.py` and `game/game_engine.py` need to be changed for Task 1. The existing architecture remains intact.

Done. 

Today PM

Task 1 is complete. Now implement only Task 2: a persistent

Add counters for:

- X
- O

The scoreboard must remain visible and unchanged when only the current round is restarted. It should reset to zero only when the user explicitly chooses to reset the whole match.

Please inspect the existing architecture and modify the minimum necessary files. Do not implement Task 3 or Task 4 yet. Explain the changes before providing the final code.

Worked for 21s

Yes. I inspected the current architecture again before making the Task 2 changes. The important point is that **Task 1 is already part of the baseline**: diagonal detection, winner-before-draw, and ignoring clicks after a round ends must remain intact. `</> game_engine`

What needs to change for Task 2

Only 2 files need changes:

1. `game/game_engine.py`

This is where the game state is stored, so the scoreboard belongs here.

We will add:

`</> Python`

```
self.x_wins = 0
self.o_wins = 0
self.draws = 0
```

We also need to separate **round reset** from **match reset**.

Currently, pressing `R` calls:

`</> Python`

```
self.__init__()
```

That would incorrectly reset the scoreboard every time. The existing `R` behavior is currently the single round-reset mechanism. `</> game_engine`

So we will introduce:

`</> Python`

```
_reset_round()
```

which resets only:

- `board`
- `current player`
- `round_over`
- `winner`

The scoreboard remains untouched.

And:

```
</> Python  
  
reset_match()
```

which resets the scoreboard as well.

Important: We will not add a separate UI control for `reset_match()` yet, because the separate restart/match-reset controls belong to **Task 4**. The method is prepared for Task 4, but Task 4's UI is not implemented now.

2. `game/renderer.py`

The existing renderer already owns the Pygame drawing code, so the scoreboard should be displayed there rather than putting additional rendering logic into `main.py`. The current renderer has the board, turn text, and banner drawing functions. `</> renderer`

We'll add:

```
</> Python  
  
draw_scoreboard()
```

and call it from `GameEngine.draw()`

Exact Task 2 changes

1. `game/game_engine.py`

Use this updated version. It keeps the Task 1 fixes and adds only the scoreboard functionality.

```
</> Python  
  
"""  
GameEngine: owns the board, turn state, round-end logic,  
and persistent scoreboard.  
  
You (the player) always play X and click to move. The computer always
```

```
plays 0 and moves automatically right after you, using a simple
random-move AI (see game/ai.py).
```

```
"""
```

```
from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move
```

```
HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'
```

```
class GameEngine:
    def __init__(self):
        self.x_wins = 0
```

2. game/renderer.py

Keep the existing renderer and add the new `draw_scoreboard()` function.

Here is the complete updated file:

</> Python

```
"""
```

```
renderer: all pygame drawing lives here, kept separate from game logic.
"""
```

```
import pygame
```

```
WIDTH, HEIGHT = 400, 500
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
WINDOW_SIZE = (WIDTH, HEIGHT)
```

```
COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
```

```
def board_pos_to_cell(pos):
    x, y = pos
```

What happens now

Suppose the game starts:

</> Plain text

X Wins: 0 O Wins: 0 Draws: 0

If X wins:

</> Plain text

X Wins: 1 O Wins: 0 Draws: 0

Press R :

</> Plain text

X Wins: 1 O Wins: 0 Draws: 0

The board is empty and a new round starts, but the scoreboard remains.

If O wins the next round:

</> Plain text

X Wins: 1 O Wins: 1 Draws: 0

Press R again:

</> Plain text

X Wins: 1 O Wins: 1 Draws: 0

If the next round is a draw:

</> Plain text

X Wins: 1 O Wins: 1 Draws: 1

Why `reset_match()` is separate

This is the important architectural change:

</> Python

```
def _reset_round(self):
```

resets **only the current round**.

Whereas:

</> Python

```
def reset_match(self):  
    self.x_wins = 0  
    self.o_wins = 0  
    self.draws = 0  
    self._reset_round()
```

resets **the complete match**.

For now, `R` calls:

</> Python

```
self._reset_round()
```

instead of:

</> Python

```
self.__init__()
```

so the scoreboard cannot accidentally be erased.

The **actual separate UI control for `reset_match()`** is intentionally not added, because that is part of **Task 4**.

What is NOT implemented

Feature	Status
Task 1 — Win/draw detection	✓ Preserved
Task 2 — X/O/Draw scoreboard	✓ Implemented
Score persists across R	✓
Match reset method	✓ Added
Separate match-reset UI	✗ Task 4
Occupied-cell validation	✗ Task 3
First-player choice	✗ Task 4
Separate restart controls	✗ Task 4

Minimum files changed: `game/game_engine.py` and `game/renderer.py`. `main.py`, `rules.py`, and `ai.py` do not need changes for Task 2. The existing `main.py` already sends keyboard events to `GameEngine`, so no new event-loop code is necessary. `</>` `main`

Implement only Task 3.

Currently, clicking an already occupied Tic-Tac-Toe cell can overwrite the existing symbol.

Fix this so that clicking an occupied cell is completely rejected.

When an occupied cell is clicked:

- the board must remain unchanged,
- the current player must remain unchanged,
- no computer move should be triggered,
- the game state must remain unchanged.

Do not modify the scoreboard or first-player/restart features.
Preserve the changes from Tasks 1 and 2.

Worked for 8s

I inspected the project structure and the existing `handle_click()` logic. The Task 3 fix requires only **one file**: `game/game_engine.py`

The current architecture puts user move handling inside `GameEngine.handle_click()` , and the existing code writes directly into the selected board cell before checking anything else. `</> game_engine`

Task 3 change

We only need to add an occupied-cell check **after converting the mouse position to a board cell and before modifying the board.**

Current code

`</> Python`

```
row, col = cell

self.board[row][col] = self.current_player

self.check_round_end()

self.current_player = 'O' if self.current_player == 'X' else 'X'
self._maybe_take_computer_turn()
```

Replace it with

`</> Python`

```
row, col = cell

if self.board[row][col] is not None:
    return

self.board[row][col] = self.current_player

self.check_round_end()

self.current_player = 'O' if self.current_player == 'X' else 'X'
self._maybe_take_computer_turn()
```

Final `handle_click()`

Using the **Task 1 + Task 2 version** as the baseline, the function should be:

`</>` Python

```
def handle_click(self, pos):
    if self.round_over:
        return

    if self.current_player != HUMAN_SYMBOL:
        return

    cell = board_pos_to_cell(pos)
    if cell is None:
        return

    row, col = cell

    # Task 3: reject occupied cells.
    if self.board[row][col] is not None:
        return

    self.board[row][col] = self.current_player

    self.check_round_end()
```

Why this satisfies Task 3

When the user clicks an occupied cell:

`</>` Python

```
if self.board[row][col] is not None:
    return
```

executes immediately.

Therefore:

- ☒ The existing symbol is not overwritten.
- ☒ `current_player` does not change.
- ☒ `check_round_end()` is not called.
- ☒ `_maybe_take_computer_turn()` is not called.

-  The scoreboard is not modified.
-  Task 1's `round_over` protection remains.
-  Task 2's persistent scoreboard remains unchanged.
-  No first-player or restart features are changed.

Example

If the board is:

 Plain text

```
x | o | .
-----
. | x | .
-----
o | . | .
```

and it is X's turn, clicking the existing `x` or `o` cell simply does nothing.

The state remains exactly the same.

Files

Changed:

- `game/game_engine.py`

Not changed:

- `game/rules.py`
- `game/renderer.py`
- `game/ai.py`
- `main.py`

Task 4 remains untouched.

One note: the Library copy currently available to me still contains the original pre-Task-2

`game_engine.py` ; I therefore based the exact change above on the **Task 2 version we established in the previous turn**, rather than accidentally reverting your scoreboard changes. The original architecture confirms that `handle_click()` is the correct place for this validation.  `game_engine`

Implement only Task 4.

Add a way for the player to choose who starts the next round: X or O.

Since the computer is O:

- if X starts, the player should make the first move;
- if O starts, the computer should automatically make the first move.

Also provide two separate controls:

1. Restart Round — resets only the current board and keeps the scoreboard.
2. Reset Match — resets the board and clears the scoreboard.

The selected starting player should take effect on the next round.

Preserve all fixes from Tasks 1, 2 and 3. Do not remove existing functionality.

Worked for 13s

I inspected the project structure and preserved the functionality from **Tasks 1, 2, and 3**. Task 4 requires changes in **three files** because it introduces new user controls.

Files to change

1. `game/game_engine.py`

- Store the selected starting player.
- Restart a round using the selected starter.
- Automatically make the computer's first move when O starts.
- Keep the scoreboard during a round restart.
- Clear the scoreboard only with **Reset Match**.
- Preserve occupied-cell validation and Task 1 win/draw logic.

2. `game/renderer.py`

- Draw:
 - Start with X / Start with O controls
 - Restart Round button
 - Reset Match button
 - Existing scoreboard and board

3. `main.py`

- Route mouse clicks to the new controls instead of treating every click as a board move.

Pygame's `Rect.collidepoint()` is appropriate for detecting whether a mouse click falls inside a button.  Pygame

1. game/game_engine.py

Replace it with this version:

</> Python

```
"""
GameEngine: owns the board, turn state, round-end logic,
persistent scoreboard, and starting-player selection.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'

class GameEngine:
    def __init__(self):
        # Persistent scoreboard
        self.x_wins = 0
        self.o_wins = 0
        self.draws = 0

        # The selected player starts the NEXT round.
```

2. game/renderer.py

Replace it with:

</> Python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 400, 560

BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
```

```
BOARD_TOP = 100

WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)

COLOR_BUTTON = (220, 220, 220)
```

3. main.py

The event loop needs to distinguish between a control click and a board click.

Replace the mouse-button section:

</> Python

```
elif event.type == pygame.MOUSEBUTTONDOWN:
    engine.handle_click(event.pos)
```

with:

</> Python

```
elif event.type == pygame.MOUSEBUTTONDOWN:
    if not engine.handle_control_click(event.pos):
        engine.handle_click(event.pos)
```

So the complete main.py becomes:

</> Python

```
"""
Tic-Tac-Toe

Run with: python main.py

Click a cell to place your symbol.
"""
```

```
import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()

    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Tic-Tac-Toe")
```

How Task 4 works

Choosing X

Click:

Next start → X

The current round **does not change**.

The selection applies to the **next round**.

When you click **Restart Round**, the new round starts with X and the player can immediately make a move.

Choosing O

Click:

Next start → O

Again, the current round is not changed.

When you click **Restart Round**, the new round starts with O.

Because O is the computer, this automatically calls:

</> Python

```
self._maybe_take_computer_turn()
```

so the computer makes the first move.

Restart Round

Click:

Restart Round

It calls:

```
</> Python
```

```
self._reset_round()
```

Therefore:

```
</> Plain text
```

Board → reset

Current round → reset

Starting player → selected X/O

Scoreboard → preserved

Reset Match

Click:

Reset Match

It calls:

```
</> Python
```

```
self.reset_match()
```

Therefore:

</> Plain text

Board → reset
Scoreboard → 0 / 0 / 0
Starting player → X
New round → starts with X

This gives the requested distinction between **round restart** and **complete match reset**.

Tasks 1–4 status

Task	Status
Task 1 — Win/draw detection	✓ Preserved
Diagonal wins	✓
Winner before draw	✓
Ignore clicks after round ends	✓
Task 2 — Scoreboard	✓ Preserved
X wins / O wins / Draws	✓
Score survives round restart	✓
Match reset clears scoreboard	✓
Task 3 — Occupied-cell validation	✓ Preserved
Occupied cell rejected	✓
Turn unchanged	✓
No computer move	✓
Task 4 — Starting player	✓ Implemented
Choose X/O for next round	✓

Task	Status
O automatically moves first	✓
Restart Round	✓
Reset Match	✓
First-player selection affects current round immediately	✗ — intentionally not done

The key detail is that **selecting X/O never changes the current board or current turn**. It only changes `starting_player` , which is used when the next round is started.